

Chapter 3 Processes

Outline

- Process concept
- Process scheduling
- Operations on Processes
- Interprocess communication
- IPC in shared-memory systems
- IPC in message-passing systems
- Examples of IPC systems
- Communication in Client-Server Systems

Objectives

- Identify the separate components of a processes and illustrate how they are represented and scheduled in an operating system
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast interprocess communication using shared memory and message passing.
- Design programs that uses pipes and POSIX shared memory to perform interprocess communication.
- Describe client-server communication using sockets and remote procedure calls.
- Design kernel modules that interact with the Linux operating system.

Process Concept

An operating system executes a variety of programs that run as a process. Process is a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process.

Multiple parts

- The program code, also called text section.
- Current activity including program counter, process registers.
- Stack containing temporary data, including function parameters, return addresses, local variables.
- Data section containing global variables

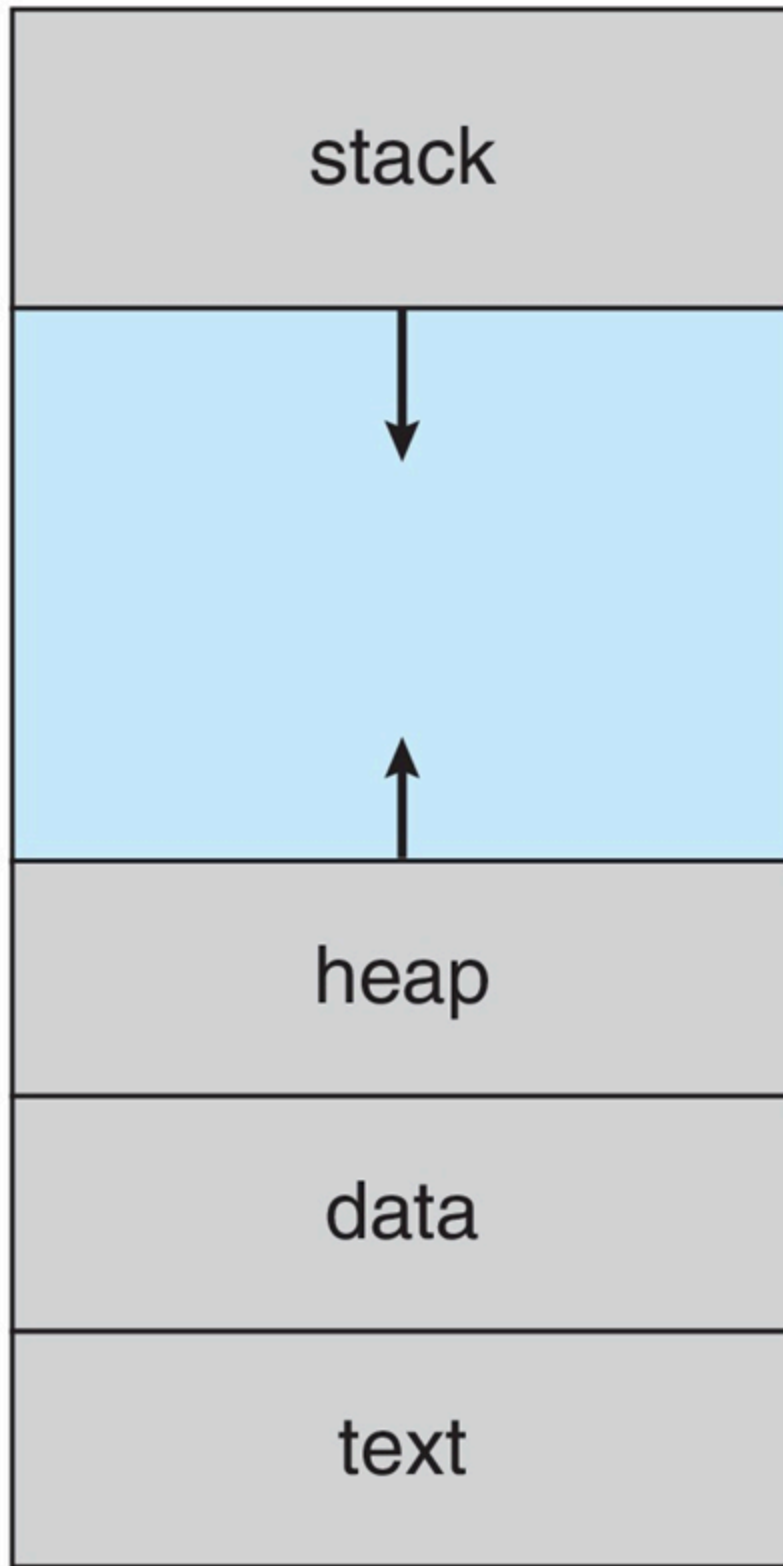
- **Heap** containing memory dynamically allocated during run time.

Differences between program and process:

- Program is **passive** entity stored on disk(executable file) while process is **active**.
- Program becomes process when an **executable file is loaded into memory**.

Execution of program started via GUI mouse clicks, command line entry, etc. One program can be **several processes**, considering multiple users executing the same program.

max



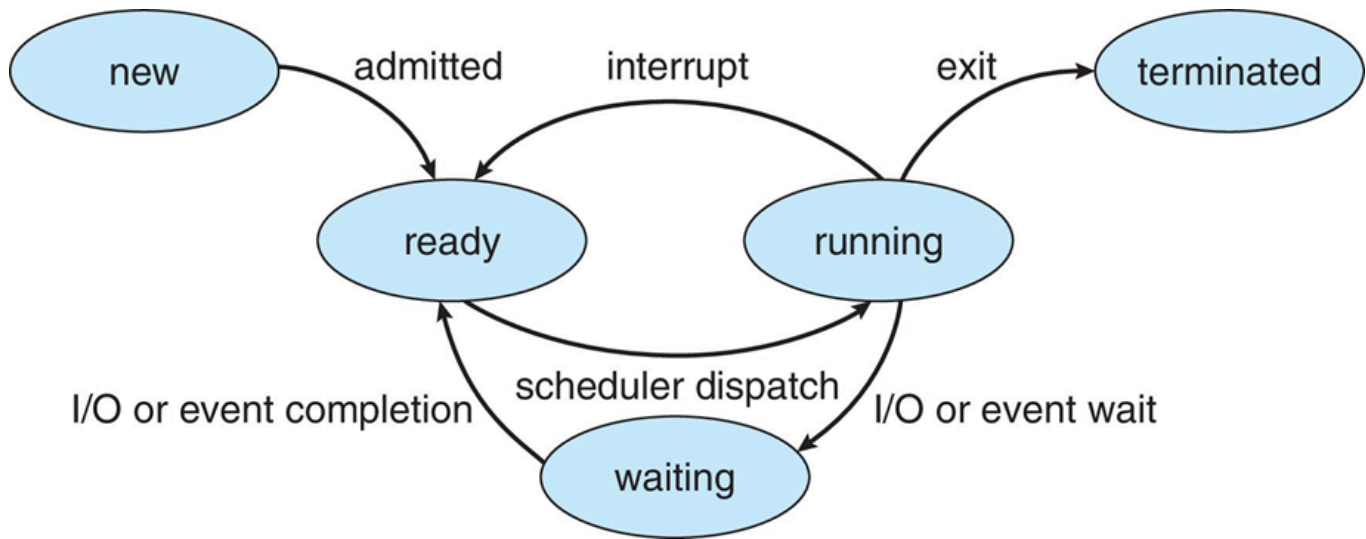
0

Process State

As a process executes, it changes **state**:

- New: the process is being created
- Running: Instructions are being executed

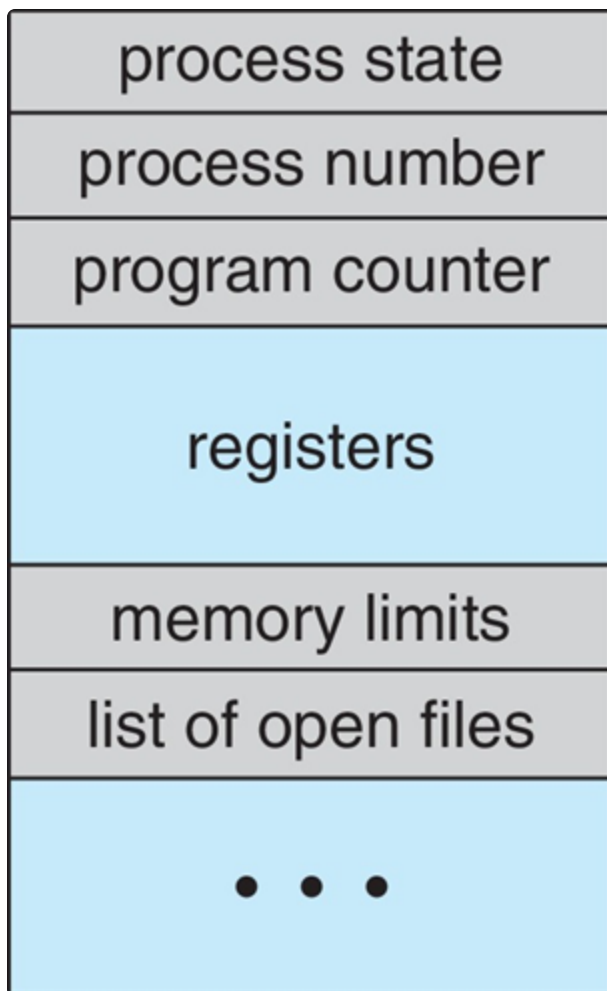
- Waiting: the process is waiting for some event to occur
- Ready: the process is waiting to be assigned to a processor
- Terminated: The process has finished execution



Process Control Block(PCB)

It is information associated with each process(also called **task control block**)

- Process state - running, waiting, etc.
- Program counter - location of instruction to next execute
- CPU registers - contents of all process-centric registers
- CPU scheduling information - priorities scheduling queue pointers
- Memory-management information - memory allocated to the process
- Accounting information - CPU used, clock time elapsed since start, time limits
- I/O status information - I/O devices allocated to process, list of open files.



Threads

So far, process has a single thread of execution. If considering **having multiple program counters per process**, then multiple locations can execute at once, corresponding multiple **threads**. Then PCB must store more details of thread and multiple program counters.

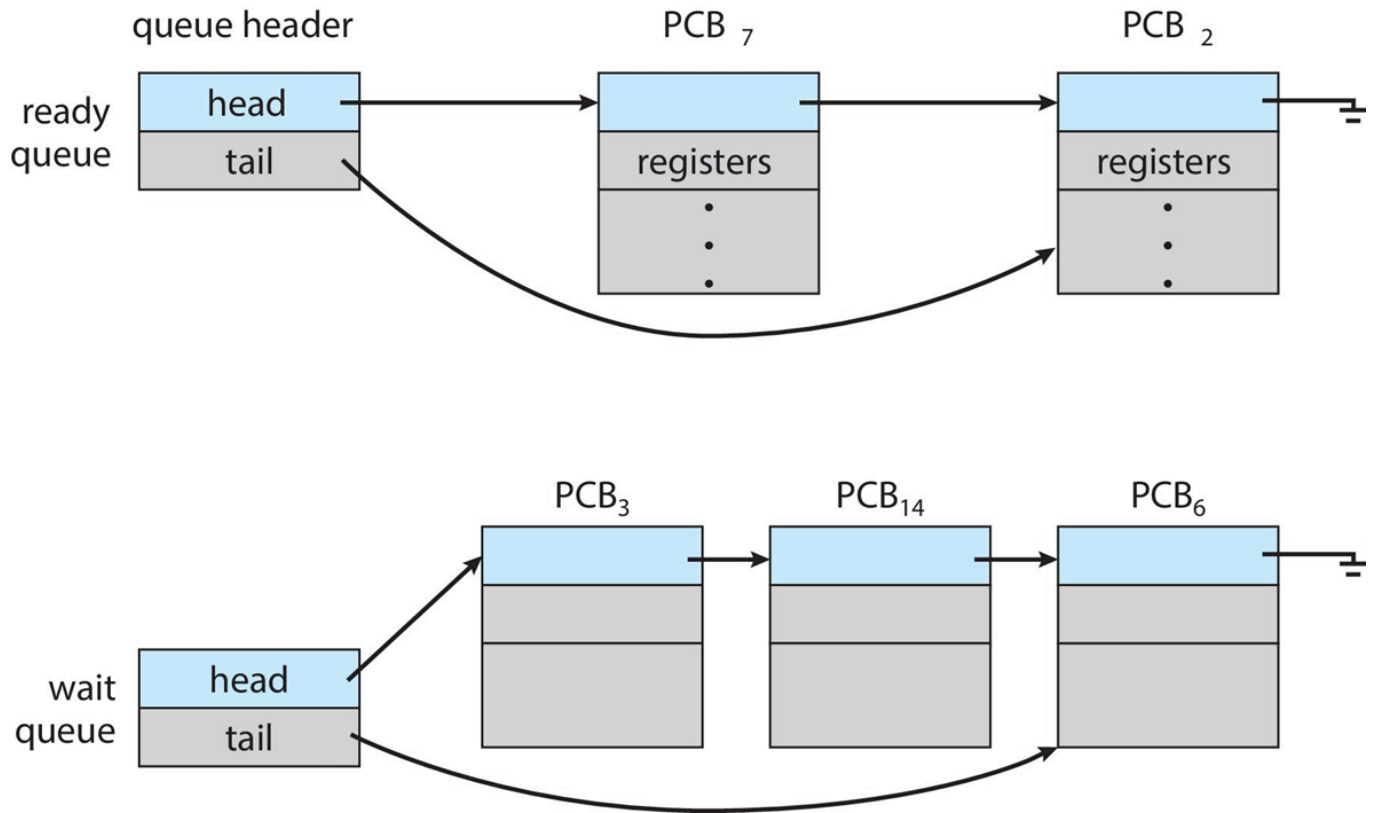
Process Representation in Linux

Process representation in Linux is represented by the C structure `task_struct`

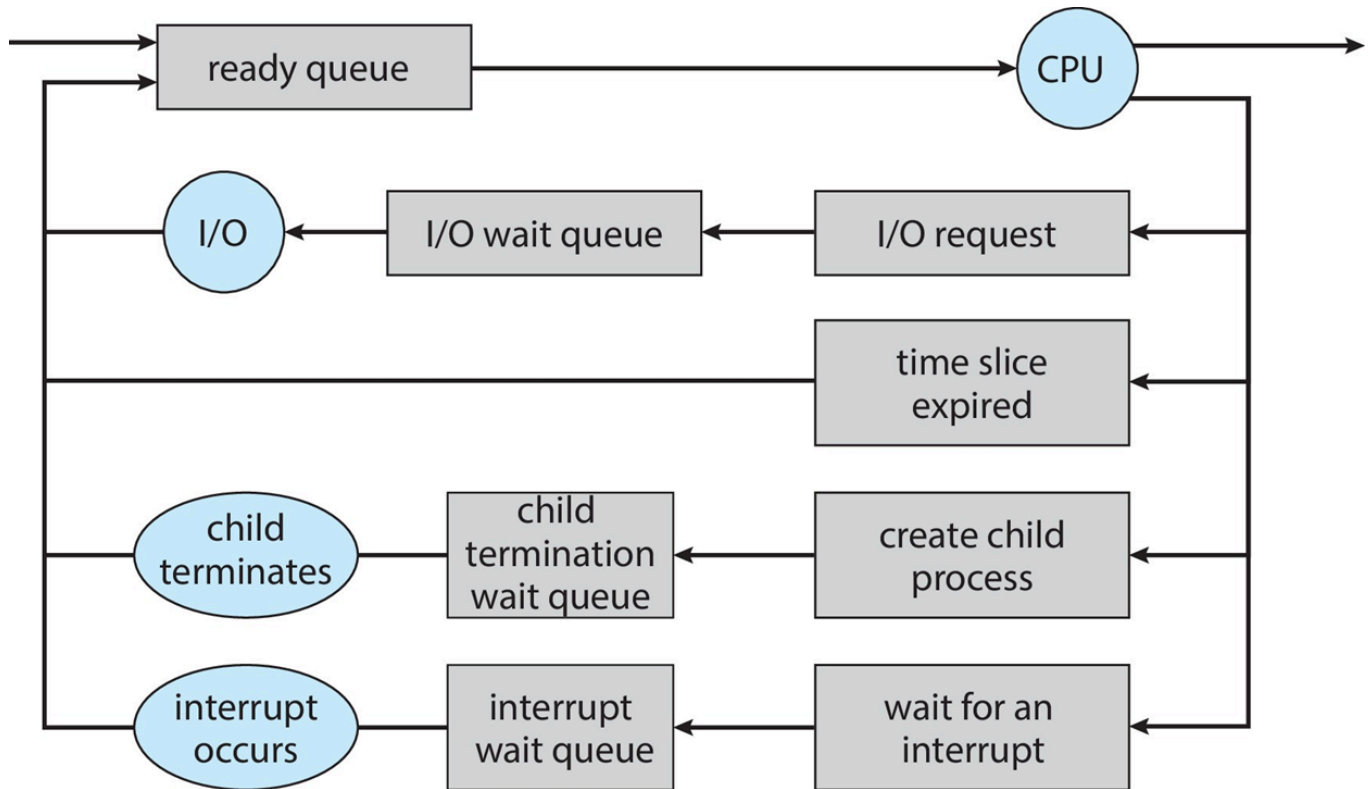
```
pid t_pid;    /* process identifier */
long state;   /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```

Process Scheduling

- **Process scheduler** selects among **available processes** for next execution on CPU core. It is to maximize CPU use and quickly switch processes onto CPU core.
- It maintains **scheduling queues** of processes
 - **Ready queue** - set of all processes residing in main memory, ready and waiting to execute.
 - **Wait queue** - set of processes waiting for an event(i.e., I/O)
 - Processes migrate among the various queues.



The process scheduling



CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.

Context Switch

When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via **context switch**.

- **Context** of a process represented in the PCB.
- Context-switch time is **pure overhead**. The system **does no useful work while switching**. The **more complex the OS and the PCB** are, the longer the context switches. It is also dependent on **hardware support**. (Some hardware provides multiple sets of registers per CPU, can load multiple contexts at once)

Multitasking in Mobile Systems

Some mobile systems allow only one process to run, like early version of iOS, others suspended.

The principles of multitasking for iOS:

- **Single foreground process** - controlled via user interface
- **Multiple background processes** - in memory, running but not on the display and with limits.

- Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback.

Android is with fewer limits:

- Background process uses a `service` to perform tasks.
- Service can keep running even if background process is suspended.
- Service has no user interface, small memory use.

Operations on Processes

System must provide mechanisms for

- Process creation
- Process termination

Process Creation

- `Parent` process create `children` processes, which create other processes, forming a `tree of processes`.
- Generally, process identified and managed via a `process identifier(pid)`.
- Resource sharing options
 - Parent and children `share all resources`
 - Children share `subset of parent's resources`
 - Parent and child `share no resources`.
- Execution options
 - Parent and children `execute concurrently`
 - Parent `waits until children terminate`
- Address space options
 - Child `duplicate` of parent.
 - Child has a `new program` loaded into it.
- Unix examples
 - `fork()` system call `creates new process`
 - `exec()` system call used after a `fork()` to replace the process' memory space with a new program
 - Parent process calls `wait()` waiting for the child to terminate.

Process Termination

- Process terminates by the `exit()` system call.(actively)
- Child process `returns status data(PID)` to parent via `wait()`

- If no parent waiting, the child process is a **zombie**
- If parent terminated without invoking `wait()`, process is an **orphan**
- Parent may **terminate the execution of children processes** using the `abort()` system call.

Some reasons like:

- Child has **exceed allocated resources**
- Task assigned to child is **no longer required**
- **The parent is exiting** and the OSs does not allow a child to continue if parent terminates. All children, grandchildren, etc. are terminated.(called **cascading termination**, initiated by the OS)

After terminating, process' resources are deallocated by OS.

Android Process Importance Hierarchy

The priority from most to least:

- Foreground process
- Visible process
- Service process
- Background process
- Empty process

Android will begin terminating processes that are least important.

Multiprocess Architecture - Chrome Browser

It is multiprocess with three different types of processes:

- Browser process
manages user interface, disk and network I/O
- Renderer process
renders web pages, deal with HTML, Javascript. A new renderer created for each website opened runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
- Plug-in process
For each type of plug-in.

Interprocess Communication

Process within a system may be **independent** or **cooperating**. Cooperating process can affect or be affected by other processes, including sharing data.

Reasons for cooperating processes:

- Information sharing
- Computation speedup
- Modularity
- Convenience

Cooperating processes need **interprocess communication(IPC)**(two models, **shared memory** and **message passing**)

Producer-Consumer Problem

Producer process produces information that is consumed by a **consumer process**. This is a **paradigm** for cooperating processes.

Two variations:

- **Unbounded-buffer**
no practical limit on the size of buffer. Producer **never waits** and consumer **waits** if there is **no buffer to consume**
- **Bounded-buffer**
there is a fixed buffer size. Producer **must wait if all buffers are full** and consumer waits if there is **no buffer to consume**.

IPC - Shared Memory

An area of memory shared among the processes that wish to communicate. The communication is **under the control of the users processes** **not the OS**.

Major problem is to provide mechanism that will allow the user processes to **synchronize their actions when they access shared memory**.

Producer process:

```
while (true) {    /* produce an item in next produced */

    while (counter == BUFFER_SIZE)

        ;    /* do nothing */

    buffer[in] = next_produced;

    in = (in + 1) % BUFFER_SIZE;

    counter++;
```

```
}
```

Consumer process:

```
while (true) {  
  
    while (counter == 0)  
  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
  
    out = (out + 1) % BUFFER_SIZE;  
  
    counter--;  
  
    /* consume the item in next consumed */  
  
}
```

Race Condition

The change of counter could be implemented by registers. If two processes simultaneously changes the counter, it may cause **difference to expected result** due to race condition.

For example, producer and consumer :

S0: producer execute register1 = counter {register1 = 5}
S1: producer execute register1 = register1 + 1 {register1 = 6}
S2: consumer execute register2 = counter {register2 = 5}
S3: consumer execute register2 = register2 - 1 {register2 = 4}
S4: producer execute counter = register1 {counter = 6}
S5: consumer execute counter = register2 {counter = 4}

The result of counter is 4 rather than expected $5+1-1=5$.

IPC-Message Passing

Processes communicate with each other **without resorting to shared variables**. IPC facility provides two operations:

- send(message)
- receive(message)

The size of message size is either fixed or variable.

If processes P and Q wish to communicate, they need to

- Establish a **communication link** between them
- Exchange messages via send and receive

Implementation of Communication Link

- Physical
 - Shared memory
 - hardware bus
 - Network
- Logical
 - Direct or indirect
 - Synchronous or asynchronous
 - Automatic or explicit buffering

Direct Communication

- Processes must name each other explicitly
 - `send(P, message)` : send a message to process P
 - `receive(Q, message)` : receive a message from process Q
- Properties of communication link
 - Link are established **automatically**
 - A link is associated with **exactly one pair of communicating processes**
 - **Between each pair** there exists **exactly one link**
 - The link may be unidirectional but is **usually bi-directional**.

Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has **a unique id**.
 - Processes can communicate only if they **share a mailbox**.
- Properties of communication link
 - Link established only if **processes share a common mailbox**
 - A link **may be associated with many processes**.
 - Each pair of processes may share **several communication links**
 - Link may be **unidirectional or bi-directional**.
- Operations
 - Create a new mailbox(port)
 - Send and receive messages from the mailbox

- delete the mailbox

Primitives are defined as

- `send(A, message)` - send a message to mailbox A
- `receive(A, message)` - receive a message from mailbox A

Synchronization

- **Blocking** is considered **synchronous**
 - Blocking send: the sender is blocked until the message is received
 - Blocking receive: the receiver is blocked until a message is availableIf both send and receive are blocking, we have a **rendezvous**.
- **Non-blocking** is considered **asynchronous**
 - Non-blocking send: the sender sends the message and continue.
 - Non-blocking receive: the receiver receives a valid message or null message

It is possible for different combination.

Producer-Consumer: Message Passing

Producer:

```
message next_produced;

while (true) {    /* produce an item in next_produced */

    send(next_produced);

}
```

Consumer:

```
message next_consumed;

while (true) {

    receive(next_consumed);

    /* consume the item in next_consumed */ } }
```

The buffer is implemented in one of three ways

- Zero capacity: no messages are queued on a link. **Sender must wait for receiver.** (rendezvous)
- Bounded capacity: finite length of n messages. **Sender must wait if link full.**
- Unbounded capacity: infinite length. **Sender never waits.**

Example of IPC Systems

POSIX: shared memory segment

Mach: messages passing

Windows: messages passing via advanced local procedure call facility

Pipes

Pipes is the conduit allowing two processes to communicate.

Issues:

- Unidirectional or bidirectional
- Half or full-duplex (in two-way communication, can simultaneously send and receive or not)
- Exist a relationship between communicating processes or not
- Used over a network or not

Types:

- Ordinary pipes: only access by the process created it. Typically used as the pipe of communication between parent process and child process
- Named pipes: can be accessed without a parent-child relationship

Ordinary Pipes

- Producer writes to one end (write-end)
 - Consumer reads from the other end (read-end)
 - Ordinary pipes are therefore **unidirectional**
 - Require **parent-child relationship** between communicating processes
- Called **anonymous pipes** in Windows

Named Pipes

- Communication is **bidirectional**
 - Parent-child relationship is not necessary
 - **Several processes** can use the named pipe
- Provided on both UNIX and Windows systems

Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls

Sockets

A socket is defined as an **endpoint** for communication.

- Composed of **IP address and port**, a number included at the start of message packet to **differentiate network services on a host**. (e.g. 161.25.19.8:1625 refers to port 1625 on host 161.25.19.8)
- Communication consists between a pair of sockets.
- All ports below 1024 are **well known**, used for standard services. Special IP address 127.0.0.1(**loopback**) to refer to system on which process is running

Sockets in Java

Three types of sockets:

- Connection-oriented(TCP)
- Connectionless(UDP)
- Multicast Socket

Remote Procedure Calls

Remote procedure call(RPC) **abstracts procedure calls between processes on networked systems**. It also uses ports for service differentiation.

- Stub: **client-side proxy** for the actual procedure on the server
- The steps:
 - The client-side stub locates the server and **marshalls** the parameters
 - The server-side stub **receives** the message, **unpacks** the marshalled parameters, and **performs** the procedure on the server.
- Data representation handled via **External data representation(XDL)** format to account for different architecture(big-endian and little-endian)
- Remote communication has more failure scenarios than local. Messages can be delivered **exactly once rather than at most once**.
- OS typically provides a rendezvous(or **matchmaker**) service to connect client and server